

Extended Test Suite — Auron Edge Cases & Security

File Inventory

File	Purpose	Size
tests/fixtures/edge_cases.yaml	YAML test definitions for all 8 groups	22 KB
tests/integration/test_edge_cases.py	pytest integration suite (TC-INT-001–053)	17 KB
tests/security/test_prompt_injection.py	pytest security suite (TC-SEC-001–023)	22 KB
tests/conftest.py	Shared fixtures, mock LLM factory, snapshot writers	11 KB
tests/fixtures/mock_responses.py	Standalone MockLLMClient + MockResponse registry	8.7 KB
tests/fixtures/injected_tool_result.json	Indirect injection payload fixture (TC-SEC-005)	154 B
requirements-test.txt	All test dependencies pinned	—
.github/workflows/extended-test-suite.yml	5-job CI pipeline (P0 gate → full → security → telemetry → nightly)	7 KB

Test ID → Description → Priority Mapping

Test ID	Group	Description	Priority
TC-INT-001	Ambiguous Inputs	Dual keyword — single tag wins via confidence threshold	P0
TC-INT-002	Ambiguous Inputs	Equal confidence tiebreak (first-registered wins)	P0
TC-INT-003	Ambiguous Inputs	Keyword overlap — amplifier suppressed without active tag	P1
TC-INT-004	Ambiguous Inputs	Negation token suppresses associated tag	P1
TC-INT-010	Amplifier Stacking	Stack 3 amplifiers within limit — no overflow	P0
TC-INT-011	Amplifier Stacking	Stack overflow guard — caps at MAX_STACK, no exception	P0
TC-INT-012	Amplifier Stacking	Amplifier queued (not applied) when no active tag	P0
TC-INT-013	Amplifier Stacking	Amplifier persists across turns within same session	P1

Test ID	Group	Description	Priority
TC-INT-014	Amplifier Stacking	Amplifier stack clears on tag switch	P1
TC-INT-020	Combined Triggers	Single message activates tag + amplifier simultaneously	P0
TC-INT-021	Combined Triggers	Multiple tags + single amplifier — attaches to winner only	P0
TC-INT-022	Combined Triggers	Amplifier hint injected exactly once into system prompt	P1
TC-INT-030	Embedding Fallback	Fallback fires when keyword confidence < threshold	P0
TC-INT-031	Embedding Fallback	No embedding call when keyword confidence $\geq$ threshold	P0
TC-INT-032	Embedding Fallback	Semantic synonym resolved to correct tag via embedding	P1
TC-INT-033	Embedding Fallback	Below-floor embedding $\rightarrow$ default tag, no exception	P2
TC-INT-040	Snapshot Restore	Valid snapshot restores state exactly	P0
TC-INT-041	Snapshot Restore	Corrupted snapshot returns SNAPSHOT_CORRUPT, no crash	P0
TC-INT-042	Snapshot Restore	Future schema version emits SCHEMA_MISMATCH warning	P1
TC-INT-043	Snapshot Restore	Orphaned amplifier dropped and logged	P1
TC-INT-050	Telemetry	tag_switch_rate emitted and in [0.0, 1.0]	P0
TC-INT-051	Telemetry	amplifier_activation_rate = activations / turns (≈ 0.4)	P0
TC-INT-052	Telemetry	prompt_injection_count increments per detected attempt	P0
TC-INT-053	Telemetry	All metrics emitted even at zero value	P1
TC-SEC-001	Injection Detection	Ignore-previous-instructions variants — parametrized $\times 4$	P0
TC-SEC-002	Injection Detection	Role-override attacks (DAN, unrestricted persona) — $\times 4$	P0
TC-SEC-003	Injection Detection	System-prompt exfiltration attempts — $\times 5$	P0
TC-SEC-004	Injection Detection	Base64-encoded injection payload — $\times 3$	P0
TC-SEC-005	Injection Detection	Indirect injection via mocked tool result	P1

Test ID	Group	Description	Priority
TC-SEC-006	Injection Detection	Unicode homoglyph bypass (Cyrillic, Greek, mixed) — × 3	P1
TC-SEC-010	Prompt Leakage	System prompt never appears in any response — × 5	P0
TC-SEC-011	Prompt Leakage	Amplifier configuration not exposed — × 4	P0
TC-SEC-012	Prompt Leakage	Tag routing dictionary not exposed — × 4	P0
TC-SEC-013	Prompt Leakage	Session snapshot JSON not exposed — × 4	P1
TC-SEC-014	Prompt Leakage	No leakage across 10-turn mixed conversation	P1
TC-SEC-020	Combined SEC	Injection + exfil in one payload — both blocked	P0
TC-SEC-021	Combined SEC	Nested base64-encoded exfiltration request	P0
TC-SEC-022	Combined SEC	JSON-wrapped injection detected	P1
TC-SEC-023	Combined SEC	Injection count reflected correctly in telemetry	P1
Total:	39 test IDs	22 P0, 15 P1, 2 P2. Parametrized cases expand to ~55 individual test instances.	

Mock LLM Architecture

The mock system has three layers:

1. **MockResponse dataclass** (`tests/fixtures/mock_responses.py`)

Mirrors every attribute the real Session/Router reads from an LLM response — `routed_tag` , `amplifier_stack` , `fallback_triggered` , `injection_detected` , `system_prompt` , etc. No monkey-patching required.

2. **FIXTURE_REGISTRY**

Keyed dictionary of canned responses referenced directly from `edge_cases.yaml` via `mock_response_fixture:` . Each fixture is a realistic, semantically correct response for its scenario.

3. **MockLLMClient**

Drop-in replacement for the real LLM client with `complete()` and `embed()` methods. Tracks `call_count` and `embed_call_count` for assertions like TC-INT-031 (`embedding_calls == 0`).

Python

```
# Instantiate in any test or notebook
from tests.fixtures.mock_responses import MockLLMClient

client = MockLLMClient()
resp = client.complete("URGENT billing issue",
    fixture="mock_billing_urgent_response")
print(resp.routed_tag)                # "billing"
print(resp.amplifier_stack)            # ["urgency"]
print(resp.prompt_includes_amplifier_hint) # True
```

Setup / Teardown Pattern

Every integration test uses the `reset_state` autouse fixture (in `conftest.py`):

Python

```
@pytest.fixture(autouse=True)
def reset_state(session, amp_stack, telemetry):
    yield                # test runs here
    session.reset()      # teardown: wipe session state
    amp_stack.clear()    # wipe amplifier stack
    telemetry.reset()    # wipe metric counters
```

Note: Security tests use a module-scoped session (fast) with per-test `session.reset()` in their own autouse fixture.

CI Commands

Bash

```
# — Install dependencies —————
pip install -r requirements-test.txt

# — Local development runs —————

# All P0 tests only (fast gate, ~30 s)
pytest tests/ -m "p0" --mock-llm -x -q

# Integration tests only
pytest tests/integration/test_edge_cases.py --mock-llm -v

# Security tests only
pytest tests/security/test_prompt_injection.py --mock-llm -v

# Telemetry tests only
pytest tests/integration/test_edge_cases.py::TestTelemetry --mock-llm -v

# Specific test by ID
pytest tests/integration/test_edge_cases.py -k "TC_INT_011" --mock-llm -v
```

```
# Run a full suite with coverage
pytest tests/ --mock-llm --cov=auron --cov-report=term-missing -q

# Parallel execution (4 workers)
pytest tests/ --mock-llm -n 4 -q

# — CI environment variables —————
MOCK_LLM=1      # enables mock LLM without --mock-llm flag
AUDIT_LOG=1     # writes logs/security_audit.jsonl per detected injection

# — Nightly (all priorities, HTML report) —————
pytest tests/ --mock-llm --cov=auron --cov-report=html:reports/htmlcov \
  --cov-fail-under=80 --junitxml=reports/nightly.xml -q
```

🔑 Key Design Decisions

- **--mock-llm flag** in `pytest.ini` adopts is intentionally not set globally — real-LLM runs remain opt-in for developer machines. CI always sets `MOCK_LLM=1`.
- `InjectionDetector.scan()` is called before `session.process()` in security tests — this validates the detection layer independently of the response guard, giving two independent assertion points per attack.
- `LeakageGuard.audit()` runs after `session.process()` — a sentinel-based post-hoc check that acts as a regression safety net separate from the system's own filtering.
- **Snapshot fixtures** are written to a `tmp_path_factory` directory at session start and exposed via `AURON_SNAPSHOT_DIR` env var, keeping the test run hermetic with no permanent filesystem side-effects.
- **CI pipeline** runs `integration-p0` first as a fast required gate before security and telemetry jobs run in parallel, minimising wall-clock time on PRs.